PHÂN CÔNG TỐI ƯU VÀ QUY TRÌNH ĐỒNG BỘ

CO2018 – Week 3: Monte Carlo π Estimation · Nhóm 5 thành viên
Mỗi approach một người · 2 người phụ trách dữ liệu và báo cáo · Đi kèm bộ mã pi-montecarlo-v2.zip

1. Vì sao cần chia lại

Cách chia trước tách Approach 2 theo dải số luồng cho hai người. Cách này có ba điểm yếu: hai người buộc phải dùng chung một máy, dữ liệu dễ lệch tại điểm nối, và người chạy đo không nhất thiết là người viết phân tích. Trong khi đó, phần chạy đo thực tế rất rẻ: toàn bộ thí nghiệm chỉ mất khoảng 5–15 phút máy chạy. Phần tốn công nhất là kiểm tra, vẽ đồ thị và viết giải thích. Phương án mới dựa trên bốn nguyên tắc:

Nguyên tắc	Áp dụng cụ thể	Lợi ích
Một lệnh chạy toàn bộ	`./scripts/run_all.sh tvX` chạy kiểm tra, đủ 3 approach, thí nghiệm 6.1 và vẽ đồ thị	Không ai phải gõ tay danh sách số luồng, không thể chạy sai tham số
Mỗi người chạy trên máy của mình	Cả 5 người cùng chạy đủ bộ thí nghiệm; mỗi máy là một bộ số liệu độc lập, không ghép	Không cần ngồi chung máy; có 5 bộ số liệu để kiểm chứng chéo và so sánh số core
Mỗi approach một chủ	TV1, TV2, TV3 mỗi người sở hữu một approach: code, kiểm thử và phần viết liên quan. TV4 lo dữ liệu và đồ thị, TV5 lo báo cáo	Người viết là người hiểu rõ phần đó nhất, trả lời được khi vấn đáp
Có cổng kiểm tra trước mỗi bước	Code freeze → kiểm tra tự động → data freeze → bản nháp → bản cuối	Lỗi bị chặn sớm, không lan sang bước sau

2. Bảng phân công mới

Ba người phụ trách ba approach, mỗi người làm trọn phần của mình từ code, kiểm thử đến phần viết trong báo cáo. Hai người còn lại phụ trách dữ liệu, đồ thị và báo cáo.

TV	Phụ trách	Code / công cụ	Phần viết trong báo cáo
TV1	**Approach 1** Single Thread	pi_single.c, common.h (timer, sinh số ngẫu nhiên, in CSV)	Thiết kế Approach 1; độ ổn định của baseline; Môi trường và phương pháp đo
TV2	**Approach 2** Local Accumulation	pi_local.c; phiên bản false sharing để đối chứng	Thiết kế Approach 2; A. Non-linear Speedup; E. Parallel Efficiency
TV3	**Approach 3** Shared + Mutex	pi_mutex.c (k = 1 và k > 1); phiên bản bỏ mutex để minh họa race condition	Thiết kế Approach 3; D. Synchronization Cost; thí nghiệm 6.1
TV4	**Đo đạc và đồ thị**	Repo và code freeze; run_all.sh, check.sh, plot.py, compare.py; gom dữ liệu 5 máy	B. Saturation Point; C. Oversubscription; thí nghiệm 6.2
TV5	**Báo cáo**	Review code của TV1–TV3 trước code freeze	Mở đầu; thí nghiệm 6.3; Best Practices; Kết luận; biên tập và nộp bài

Phần việc chung cho cả 5 người: cài Linux hoặc WSL2, chạy `make check` và `./scripts/run_all.sh tvX` trên máy mình, gửi thư mục `results/tvX` cho TV4. Mỗi người mất khoảng 30 phút cho phần này.

2.1 Chi tiết nhiệm vụ

TV1 – Approach 1: Single Thread (baseline)

- Chịu trách nhiệm `pi_single.c` và `common.h`: hàm đo thời gian, hàm sinh điểm ngẫu nhiên bằng rand_r, hàm in CSV. Đây là nền móng mà Approach 2 và 3 dùng lại.
- Làm hai thí nghiệm nhỏ minh họa mục "Important considerations" của đề: (1) thêm printf trong vòng lặp để thấy I/O làm sai lệch thời gian; (2) đo bằng clock() so với CLOCK_MONOTONIC để thấy vì sao không dùng clock() với chương trình đa luồng.
- Từ 5 máy, lập bảng T_{single} và độ lệch chuẩn để chứng minh baseline ổn định, vì mọi giá trị speedup đều chia cho con số này.
- Viết: thiết kế Approach 1, độ ổn định của baseline, mục Môi trường và phương pháp đo (cấu hình máy chính, N, seed, số lần chạy, cách đo).

TV2 – Approach 2: Local Accumulation

- Chịu trách nhiệm `pi_local.c`: chia việc cho T luồng, biến đếm cục bộ, seed riêng mỗi luồng, gộp kết quả sau join.
- Viết thêm phiên bản ghi trực tiếp vào mảng chung `counts[i]` trong vòng lặp để đo ảnh hưởng của false sharing.
- Viết: thiết kế Approach 2 (mô hình map–reduce), A. Non-linear Speedup (Amdahl's Law, chi phí tạo và join luồng, băng thông bộ nhớ), E. Parallel Efficiency.

TV3 – Approach 3: Shared Variable + Mutex

- Chịu trách nhiệm `pi_mutex.c`: biến đếm toàn cục, mutex, tham số k cho thí nghiệm 6.1.
- Chạy một lần phiên bản bỏ mutex để ghi nhận π sai do race condition (chỉ minh họa, không đưa vào bộ đo).
- Viết: thiết kế Approach 3, D. Synchronization Cost (lock contention, chi phí lock/unlock, cache line bị chuyển giữa các core), thí nghiệm 6.1.

TV4 – Đo đạc và đồ thị

- Quản lý repo, gắn tag code freeze, duy trì các script chạy và vẽ.
- Gom thư mục kết quả của 5 máy, kiểm tra π trùng nhau, commit trùng tag, độ lệch chuẩn; chọn máy chính; xuất toàn bộ đồ thị và bảng so sánh giữa các máy.
- Viết: B. Saturation Point, C. Oversubscription, thí nghiệm 6.2. Với dữ liệu nhiều máy, TV4 chỉ ra được điểm bão hòa dịch chuyển theo số core của từng máy.

TV5 – Báo cáo

- Tạo file báo cáo chung với đủ đề mục và tên người phụ trách từng mục ngay từ ngày đầu.
- Review code của TV1–TV3 theo checklist trước code freeze.
- Viết: Mở đầu, thí nghiệm 6.3 (đặt Approach 2 và 3 cạnh nhau, giải thích bằng critical section, contention, serialization), Best Practices, Kết luận.
- Ghép báo cáo, thống nhất văn phong và thuật ngữ, đối chiếu mọi con số với summary.csv, kiểm tra đủ 5 mục Deliverables rồi nộp bài.

3. Cơ chế đảm bảo chính xác

3.1 Kiểm tra tự động trước khi đo

`make check` chạy nhanh với $N = 10^6$ và kiểm tra ba điều. `run_all.sh` tự gọi lại bước này và dừng ngay nếu có lỗi, nên không thể đo bằng code sai.

- π của Approach 1 nằm trong 3.14 ± 0.01.
- Approach 2 với T = 1 cho π giống hệt Approach 1.
- Với T = 2, 3, 7, 8, 16: Approach 2, Approach 3 (k = 1) và Approach 3 (k = 100) cho π **giống hệt nhau**. T = 3 và T = 7 kiểm tra luôn phần chia dư khi N không chia hết cho T.

```
$ make check
  [OK]   Approach 1: pi = 3.144728
  [OK]   Approach 2 voi T=1 trung Approach 1
```

```
  [OK]   T=2: Approach 2 = Approach 3 (k=1) = Approach 3 (k=100) = 3.143268
  ...
check.sh: TAT CA DEU DUNG
```

3.2 Kiểm chứng chéo giữa các máy

Hàm `rand_r` của glibc là thuật toán xác định, nên cùng seed và cùng T thì π ước lượng phải **giống hệt nhau trên mọi máy** chạy Linux hoặc WSL. Khi nhận đủ 5 thư mục, TV4 so sánh cột `pi_estimate` giữa các máy. Thời gian được phép khác nhau, còn π mà khác nhau nghĩa là có người chạy sai phiên bản code. Máy macOS không dùng glibc nên π có thể khác; thành viên dùng Mac nên chạy trong máy ảo Linux hoặc nhờ máy khác.

3.3 Truy vết phiên bản code

`run_all.sh` tự ghi số core, ngày giờ, mã commit Git và phiên bản gcc vào `machine_info.txt`. TV4 kiểm tra mọi thư mục cùng một mã commit trùng với tag code freeze. Nếu nhóm không dùng Git, TV4 gửi một file zip duy nhất và mọi người chỉ dùng file đó.

3.4 Kiểm tra chất lượng số liệu

- Mỗi dòng trong `summary.csv` có độ lệch chuẩn của 5 lần chạy. Nếu độ lệch chuẩn vượt 10% thời gian trung bình ở nhiều cấu hình, máy đó đang bị nhiễu (chạy ứng dụng nền, chưa cắm sạc) và phải chạy lại.
- **Máy chính** dùng cho các đồ thị chính được TV4 chọn **sau khi có số liệu**: ưu tiên máy có nhiều core nhất trong số các máy có độ lệch chuẩn thấp. Nhờ vậy nhóm không phụ thuộc vào máy của một người cố định.
- Mọi con số xuất hiện trong báo cáo phải lấy từ `summary.csv` hoặc `results/compare/machines.csv`; TV5 đối chiếu lại từng số trước khi nộp.

4. Cơ chế đồng bộ trong nhóm

Hạng mục	Quy ước
Nguồn code duy nhất	Một repo GitHub do TV4 quản lý (hoặc một file zip duy nhất nếu không dùng Git). TV1–TV3 gửi thay đổi, chỉ TV4 merge vào nhánh main
Code freeze	Sau khi TV5 review xong, TV4 gắn tag `v1-freeze`. Từ đó không ai sửa code nữa; muốn sửa phải báo cả nhóm và chạy đo lại
Đặt tên dữ liệu	Mỗi người chạy với đúng mã của mình: tv1 … tv5. Không sửa tay file CSV, gửi nguyên thư mục `results/tvX`
Nơi lưu chung	Một thư mục Google Drive gồm `results/` (TV4 gom), `figures/` (TV4 xuất) và file báo cáo chung
Mẫu báo cáo	TV5 tạo sẵn file báo cáo với đủ đề mục và ghi tên người phụ trách từng mục; mỗi người viết thẳng vào phần của mình
Thuật ngữ	Giữ nguyên tiếng Anh: speedup, efficiency, saturation point, oversubscription, contention, critical section, context switch, false sharing
Trích dẫn số liệu	Mỗi nhận định đi kèm số hình hoặc số bảng. Đồ thị chính lấy từ máy chính; số liệu các máy khác dùng trong bảng so sánh
Kênh trao đổi	Nhóm chat hiện tại; mỗi khi qua một mốc ở mục 5, người phụ trách nhắn xác nhận kèm đường dẫn file

5. Tiến độ và các mốc kiểm tra

Các mốc tính theo ngày kể từ khi bắt đầu. Bài đã quá hạn nên kế hoạch được nén còn 4 ngày; nếu cần nhanh hơn, ngày 1 và ngày 2 có thể gộp lại.

Ngày	Mốc	Việc cần xong	Cổng kiểm tra
1	M1 – Code freeze	TV4 đưa bộ mã v2 lên repo; TV1–TV3 kiểm tra lại phần code của mình; TV5 review; cả nhóm cài WSL và chạy `make check`	5/5 người báo `TAT CA DEU DUNG`; TV4 gắn tag v1-freeze
2	M2 – Data freeze	Cả nhóm chạy run_all.sh, gửi thư mục kết quả; TV4 kiểm tra π trùng nhau, commit trùng tag, độ lệch chuẩn	Đủ 5 thư mục hợp lệ; TV4 chốt máy chính
3	M3 – Bản nháp	TV4 gửi đồ thị và bảng so sánh buổi sáng; mọi người viết phần được giao	Đủ các đoạn A–E, 6.1–6.3 trong file chung
4	M4 – Nộp bài	TV5 ghép, thống nhất văn phong, đối chiếu số liệu, kiểm tra đủ Deliverables và nộp	Đủ 5 mục Deliverables của đề

Đường găng là chuỗi M1 → M2 → đồ thị của TV4. Trong lúc chờ số liệu, TV1, TV2, TV3, TV5 viết trước phần thiết kế và lý thuyết (Amdahl's Law, cơ chế mutex, context switch, false sharing), đến ngày 3 chỉ cần chèn số liệu và hình.

6. Quy trình từng bước cho mỗi thành viên

```
# 1. Cai moi truong (Windows: mo PowerShell voi quyen admin, chay 'wsl --install', khoi dong lai)
sudo apt update && sudo apt install -y build-essential python3-pandas python3-matplotlib

# 2. Lay code (Git hoac giai nen pi-montecarlo-v2.zip)
git clone <repo cua nhom> && cd pi-montecarlo && git checkout v1-freeze

# 3. Kiem tra tinh dung - phai thay dong 'TAT CA DEU DUNG'
make check

# 4. Dong ung dung nang, cam sac, roi chay toan bo (thay tv1 bang ma cua minh)
./scripts/run_all.sh tv1

# 5. Nen va gui thu muc ket qua cho TV4
zip -r tv1.zip results/tv1

# 6. (Chi TV4) giai nen du 5 thu muc vao results/ roi so sanh
python3 scripts/compare.py      # -> results/compare/machines.csv, speedup_all_machines.png
```

`run_all.sh` tự xác định số core C của máy và chọn danh sách số luồng: Approach 2 chạy từng số nguyên từ 1 đến max(16, C), sau đó 24, 32, 48, 64, 96, 128; Approach 3 chạy 1, 2, 4, ..., 128 và C; thí nghiệm 6.1 chạy ở T = C với k = 1, 10, 100, 1 000, 10 000. Mỗi cấu hình 5 lần, $N = 10^7$, seed = 2024.

7. Tiêu chí hoàn thành của từng người

TV	Được coi là xong khi
TV1	Giải thích được từng dòng của pi_single.c và common.h; có số liệu hai thí nghiệm printf và clock(); bảng T_{single} của 5 máy; mục Phương pháp đo đầy đủ
TV2	Giải thích được từng dòng của pi_local.c; đoạn A và E có speedup, efficiency cụ thể tại ít nhất 3 mức T; có so sánh với phiên bản false sharing
TV3	Giải thích được từng dòng của pi_mutex.c; đoạn D có tỉ lệ thời gian Approach 3 / Approach 2 tại cùng T; 6.1 chỉ ra mức k từ đó thời gian gần Approach 2
TV4	Repo có tag v1-freeze; đủ 4 hình của máy chính, hình và bảng so sánh các máy; đoạn B, C nêu điểm bão hòa của từng máy so với số core
TV5	Biên bản review code; đoạn 6.3 và Best Practices dựa trên kết quả thật; mọi số trong báo cáo khớp summary.csv; bài đã nộp đủ file

8. Rủi ro và cách xử lý

Tình huống	Cách xử lý
make check báo SAI	Không chạy đo. Báo TV4 và người phụ trách approach bị lỗi, kèm toàn bộ dòng in ra; thường do sửa code sau freeze hoặc dùng nhầm phiên bản
Máy dùng macOS	Chạy trong máy ảo Linux; nếu không được thì người đó bỏ qua bước chạy, nhóm vẫn đủ dữ liệu từ các máy còn lại
WSL chỉ nhận ít core	Kiểm tra bằng nproc; WSL2 mặc định dùng toàn bộ core, nếu thấp hơn thì sửa file .wslconfig
Độ lệch chuẩn lớn	Đóng ứng dụng nền, cắm sạc, xóa thư mục results/tvX rồi chạy lại
Một thành viên trễ hạn	Mỗi máy là một bộ dữ liệu độc lập nên 4 máy vẫn đủ để phân tích; phần viết của người trễ được hỗ trợ theo cặp: ba người phụ trách approach hỗ trợ lẫn nhau, TV4 và TV5 hỗ trợ nhau
Số liệu không đúng kỳ vọng	Không sửa số. Kiểm tra lại cấu hình máy và ghi nhận hiện tượng; giải thích được kết quả bất thường cũng là điểm cộng